

# Motor de Tráfico Aéreo: Análisis y Prueba de Escritorio (Fase 1)

## 1. Documentación de Memoria: El uso de `Option::take()`

En Rust, el sistema de tipos y el "Borrow Checker" previenen tener dos referencias mutables a los mismos datos al mismo tiempo, lo que elimina errores como *data races* y punteros colgantes (dangling pointers).

Dentro del código de un árbol AVL, la estructura del nodo utiliza `Option<Box<Nodo>>` para sus hijos. Las rotaciones de árboles (simples y dobles) implican desenlazar un nodo de su padre y conectarlo a otro lugar.

**¿Qué ocurre con la propiedad (ownership) al usar `Option::take()` ?** El método `take()` extrae el valor contenido dentro del `Option`, dejando temporalmente un `None` en su lugar original. Esto es fundamental durante las rotaciones porque:

- 1. Transferencia Segura de Propiedad:** Permite mover el `Box<Nodo>` (el hijo) fuera de la estructura del nodo padre sin violar las reglas de préstamo estricto del compilador. Si intentáramos reasignar ramas directamente (ej. `padre.izquierdo = hijo.derecho`) mientras conservamos referencias vivas, el compilador lo rechazaría por posibles alias mutables.
- 2. Evita Clones Innecesarios:** `take()` toma directamente la propiedad de los datos alojados en memoria sin necesidad de clonar u duplicar el árbol entero, garantizando una alta eficiencia ( $O(1)$ ) al mover ramas en lugar de  $O(N)$ .
- 3. Manejo Seguro de Punteros Nulos:** Previene errores fatales de memoria (segfaults) si un nodo hijo no existe (`None`), garantizando siempre la consistencia dentro de las reglas de enumeraciones (`Enums`) de Rust.

---

## 2. Concepto de `Box<Nodo>`

En la definición de la estructura proporcionada se utiliza `Box<Nodo>` para los subárboles:

```
struct Nodo {  
    vuelo: Vuelo,  
    izquierdo: Option<Box<Nodo>>,  
    derecho: Option<Box<Nodo>>,  
    altura: i32,  
}
```

**¿Por qué se usa `Box<Nodo>` en lugar de `Nodo` directamente?** En Rust, el compilador necesita saber el **tamaño exacto de cada estructura en tiempo de compilación** para asignarle el espacio correcto en el *Stack* (pila de llamadas).

Un `Nodo` recursivo que contenga otro `Nodo` en su interior tendría un tamaño infinito desde el punto de vista del compilador, ya que un nodo contiene un nodo, que a su vez contiene otro nodo. Esto generaría un error de tipo "recursive type has infinite size".

Al envolver los hijos en un `Box<Nodo>`, le estamos indicando al compilador que aloje dinámicamente los datos del nodo hijo en el **Heap** (montículo). El `Box` actúa como un "Puntero Inteligente" (Smart Pointer), lo que significa que la estructura principal ahora solo almacena la *dirección de memoria* hacia el Heap (un tamaño fijo y predecible, generalmente 8 bytes). Esto soluciona la restricción geométrica del tamaño y asegura que

Rust pueda compilar de forma segura, destruyendo la memoria del nodo hijo de manera determinista cuando el `Box` sale del alcance.

---

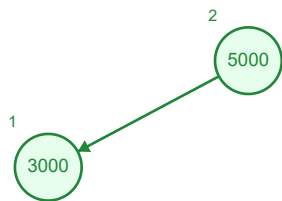
### 3. Prueba de Escritorio AVL

Altitudes a insertar: [5000, 3000, 2000, 4000, 3500, 6000]

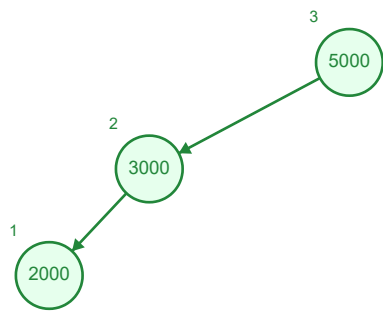
**Paso 1: Insertar 5000** El nodo se convierte en la raíz del árbol.



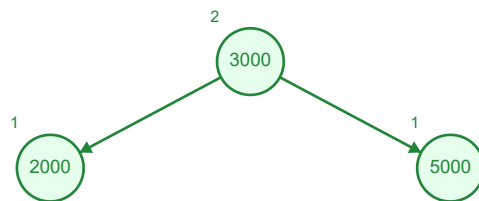
**Paso 2: Insertar 3000** El 3000 es menor que 5000, se ubica a la izquierda. Balance de la raíz = 1.



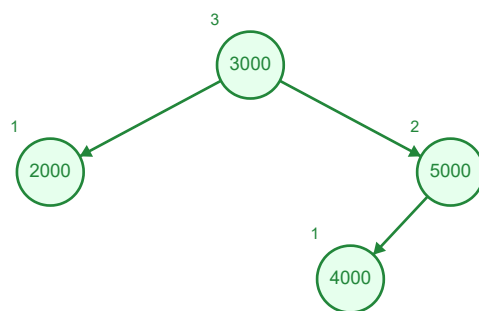
**Paso 3: Insertar 2000** El 2000 es menor que 3000. Se inserta a su izquierda. Esto provoca un desbalanceo en la raíz (5000), donde el balance es  $> 1$  (Izquierda-Izquierda).



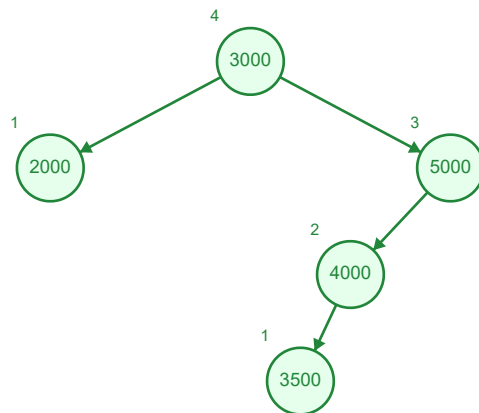
\*Se ejecuta una **\*\*Rotación Simple a la Derecha\*\*** en el nodo 5000.\* El 3000 asciende como la nueva raíz del árbol.



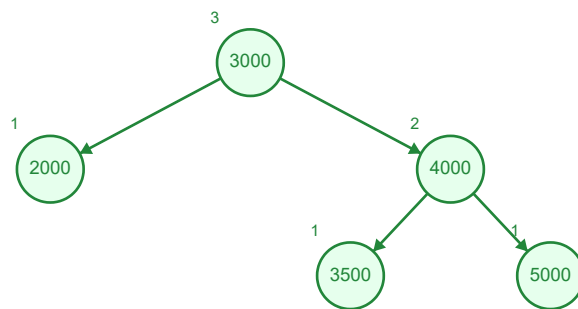
**Paso 4: Insertar 4000** Es mayor que 3000 y menor que 5000, por lo que es el hijo izquierdo de 5000.



**Paso 5: Insertar 3500** Se posiciona como el hijo izquierdo del 4000. Esto genera un desbalance en el subárbol del 5000. Estamos ante un caso Izquierda-Izquierda a nivel del nodo 5000.



\*Se realiza una **\*\*Rotación Simple a la Derecha\*\*** en el nodo 5000.\* El 4000 sube, tomando al 5000 como su hijo derecho.



**Paso 6: Insertar 6000 (Árbol Final)** El 6000 es mayor, siendo el hijo derecho del 5000. Al calcular los factores de balanceo globales, la raíz principal (3000) tiene una altura izquierda de 1 (nodo 2000) y una altura derecha de 3 (nodo 4000). Balance = -2. Se presenta un caso **Derecha-Derecha** global. Se aplica una **Rotación Simple a la Izquierda** sobre la raíz 3000. El nodo 4000 sube como nueva raíz principal, el 3000 baja a la izquierda y adopta al 3500 como su hijo derecho.

*(Nota: En algunas construcciones visuales rápidas, suele confundirse la posición del 3500 colocándose equivocadamente a la izquierda del 2000, sin embargo, matemáticamente hablando un ABB exige que  $3500 > 3000$ , lo que lo ubica estrictamente a la derecha).*

El Árbol AVL Final Matemáticamente Balanceado es el siguiente:

